

Together First, Apart Later? When Optimizers Start to Learn Different Functions

Project plan (9.520, Fall 2026)

Juntang Wang

Víctor Conchello

October 8, 2026

1 Problem

The optimizer decides which of many equal-loss solutions training returns. In linear models each first-order method converges to the max-margin solution of its own norm: ℓ_2 for gradient descent (Soudry et al., 2018), ℓ_∞ for Adam (Zhang et al., 2024a) and the spectral norm for Muon (Fan et al., 2025). In deep networks we only see the end point: adaptive methods were found to generalize worse (Wilson et al., 2017), then to learn richer features (Vasudeva et al., 2025), and on language models most optimizers reach similar loss (Zhao et al., 2025). We ask *when* the difference appears. From the same initialization and the same data order, do SGD, AdamW and Muon first follow the same path in function space and only later separate? If so, where is the split, what happens in the optimization at that moment, and is the final difference already decided there?

In vision, Jastrzebski et al. (2020) show SGD runs with different learning rates separating in prediction space, and tie the early dynamics to a *break-even point*, the first time training meets a stability boundary along the sharpest direction. We test whether the same picture holds across optimizers, on a small language model, and whether the split coincides with an edge-of-stability event (Cohen et al., 2021; Cohen et al., 2022; Andreyev and Beneventano, 2024).

2 What we plan to achieve

1. **Divergence curves.** The function-space distance between optimizers over training, compared both at equal steps and at equal training loss, so that “faster” is not confused with “different”.
2. **A located split point t^*** for each pair of optimizers, with uncertainty, defined against the seed-to-seed spread of a single optimizer.
3. **What coincides with t^* :** sharpness reaching its stability threshold, the end of warmup, or a loss level.
4. **A causal check by swapping:** swapping optimizer before t^* should end at the new optimizer’s function, and swapping after t^* should stay near the old one.
5. **Moonshot:** which layers carry the sharp direction at the edge of stability, by optimizer.

3 How we plan to achieve it

Setup. nanoGPT (Karpathy, 2022), a small decoder-only transformer, on character-level Shakespeare (minutes per run) and then TinyStories at about 10M and 30M parameters. Optimizers: SGD with momentum, AdamW, and Muon for the hidden matrices with AdamW on embeddings and head, as in Jordan et al. (2024). Each optimizer gets a short learning-rate sweep, and we keep the best rate. All runs share the initialization and the batch order. A small CNN on CIFAR-10 reproduces the vision result as a check of the pipeline.

Function distance. On a fixed held-out set of contexts we log, every few hundred steps, the next-token distributions of each run. The distance between two runs is the mean Jensen–Shannon divergence between their predictions, plus top-1 agreement. The noise floor is the distance between two runs of the *same* optimizer with different data order. We align runs in two ways: by step, and by training loss (each checkpoint matched to the other run’s checkpoint with the same loss).

Locating the split. t^* is the first point where the loss-matched distance between two optimizers exceeds the noise floor by a fixed margin over several consecutive checkpoints, with a bootstrap over seeds for its uncertainty. As a

second, basin-level test we measure linear mode connectivity (Frankle et al., 2020): the loss barrier between two runs forked from a common checkpoint with different optimizers, as a function of the fork time.

What happens at the split. Along each run we track the top Hessian eigenvalue by power iteration on a fixed batch: $\lambda_{\max}$ for SGD, the preconditioned version for AdamW (Cohen et al., 2022), and batch sharpness (Andreyev and Beneventano, 2024). We compare t^* with the step at which each reaches its threshold, and with the end of warmup.

Swap. We fork optimizer A at times before, at and after t^* and continue with B . B ’s state is accumulated in shadow during A ’s run, so B starts warm. The control is A continued. We report where each fork lands between pure A and pure B , by the distance above and by a few function metrics: input-Jacobian norm, token-substitution sensitivity (Hahn et al., 2021), and the ℓ_2 , ℓ_∞ and spectral weight norms each optimizer is predicted to keep small.

Moonshot: the anatomy of the edge of stability. At each logged step we take the top Hessian eigenvector v and split it by parameter block: embeddings, the query-key and value-output matrices and the MLP of each layer, the norms and the head. For block b we record its share of the direction, $\|v_b\|^2$, and its share of the sharpness, $v_b^\top (Hv)_b$, whose sum over blocks is $\lambda_{\max}$. The questions: which components carry the sharpness when training reaches the edge of stability, whether this moves across layers during training, and whether it differs by optimizer, given that transformer Hessians differ strongly across blocks (Zhang et al., 2024b). If the block that dominates changes near t^* , the two halves of the project meet.

4 Timeline

Oct 9–18: nanoGPT with three optimizers on Shakespeare, shadow state, prediction logging, noise floor; CIFAR-10 check. **Oct 19–30:** divergence curves, first t^* , sharpness tracking; initial checkpoint. **Oct 31–Nov 15:** TinyStories at two sizes; mode connectivity against fork time; swaps. **Nov 16–30:** statistics over seeds; moonshot. **Dec 1–4:** presentation. **Dec 5–11:** paper, code and Colab notebook.

Split of work: Juntang leads the divergence and sharpness tracking, Víctor the swaps and function metrics; the moonshot is shared.

References

- Andreyev, A. and P. Beneventano (2024). *Edge of Stochastic Stability: Revisiting the Edge of Stability for SGD*. Preprint.
- Cohen, J. M., B. Ghorbani, S. Krishnan, et al. (2022). *Adaptive Gradient Methods at the Edge of Stability*. Preprint.
- Cohen, J. M., S. Kaur, Y. Li, et al. (2021). “Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability”. In: *ICLR*.
- Fan, C., M. Schmidt, and C. Thrampoulidis (2025). “Implicit Bias of Spectral Descent and Muon on Multiclass Separable Data”. In: *NeurIPS*.
- Frankle, J., G. K. Dziugaite, D. M. Roy, and M. Carbin (2020). “Linear Mode Connectivity and the Lottery Ticket Hypothesis”. In: *ICML*.
- Hahn, M., D. Jurafsky, and R. Futrell (2021). “Sensitivity as a Complexity Measure for Sequence Classification Tasks”. In: *Transactions of the Association for Computational Linguistics*.
- Jastrzebski, S., M. Szymczak, S. Fort, et al. (2020). “The Break-Even Point on Optimization Trajectories of Deep Neural Networks”. In: *ICLR*.
- Jordan, K., Y. Jin, V. Boza, et al. (2024). *Muon: An Optimizer for Hidden Layers in Neural Networks*. Blog post. URL: <https://kellerjordan.github.io/posts/muon/>.
- Karpathy, A. (2022). *nanoGPT*. Software. URL: <https://github.com/karpathy/nanoGPT>.
- Soudry, D., E. Hoffer, M. S. Nacson, et al. (2018). “The Implicit Bias of Gradient Descent on Separable Data”. In: *JMLR*.
- Vasudeva, B., J. W. Lee, V. Sharan, and M. Soltanolkotabi (2025). “The Rich and the Simple: On the Implicit Bias of Adam and SGD”. In: *NeurIPS*.
- Wilson, A. C., R. Roelofs, M. Stern, et al. (2017). “The Marginal Value of Adaptive Gradient Methods in Machine Learning”. In: *NeurIPS*.
- Zhang, C., D. Zou, and Y. Cao (2024a). “The Implicit Bias of Adam on Separable Data”. In: *NeurIPS*.
- Zhang, Y., C. Chen, T. Ding, et al. (2024b). “Why Transformers Need Adam: A Hessian Perspective”. In: *NeurIPS*.
- Zhao, R., D. Morwani, D. Brandfonbrener, et al. (2025). “Deconstructing What Makes a Good Optimizer for Language Models”. In: *ICLR*.